

RaSA: Robust and Adaptive Secure Aggregation for Edge-Assisted Hierarchical Federated Learning

Lingling Wang¹, Member, IEEE, Mei Huang², Zhengyin Zhang³, Meng Li⁴, Senior Member, IEEE, Jingjing Wang⁵, Member, IEEE, and Keke Gai⁶, Senior Member, IEEE

Abstract—Secure Aggregation (SA), in the Federated Learning (FL) setting, enables distributed clients to collaboratively learn a shared global model while keeping their raw data and local gradients private. However, when SA is implemented in edge-intelligence-driven FL, the open and heterogeneous environments will hinder model aggregation, slow down model convergence speed, and decrease model generalization ability. To address these issues, we present a Robust and adaptive Secure Aggregation (RaSA) protocol to guarantee robustness and privacy in the presence of non-IID data, heterogeneous system, and malicious edge servers. Specifically, we first design an adaptive weights updating strategy to address the non-IID data issue by considering the impact of both gradient similarity and gradient diversity on the model aggregation. Meanwhile, we enhance privacy protection by preventing privacy leakage from both gradients and aggregation weights. Different from previous work, we address system heterogeneity in the case of malicious attacks, and the malicious behavior from edge servers can be detected by the proposed verifiable approach. Moreover, we eliminate the influence of straggling communication links and dropouts on the model convergence by combining efficient product-coded computing with repetition-based secret sharing. Finally, we perform a theoretical analysis that proves the security of RaSA. Extensive experimental results show that RaSA can ensure model convergence without affecting the generalization ability under non-IID scenarios. Moreover, the decoding efficiency of RaSA achieves $1.33\times$ and $6.4\times$ faster than the state-of-the-art product-coded and one-dimensional coded computing schemes.

Index Terms—Federated learning, privacy protection, secure aggregation, robustness, weight updating.

I. INTRODUCTION

THE past few years have witnessed significant achievements in edge computing and Artificial Intelligence (AI). Recently, edge intelligence, aiming to facilitate the deployment of AI services by edge computing, has attracted great attention and interest from researchers and the industry. It provides a promising solution to the distributed model learning requirements of massive Internet of Things devices and the increasing need for real-time data analysis, decision-making, and productivity enhancements. As the key enabler to edge intelligence, the widely adopted federated learning [1] accomplishes the distributed learning tasks at the network edge or end devices without revealing any raw data. However, when deploying FL algorithms to a practical edge intelligence system, a major hindrance is the open and heterogeneous environment that causes non-independent and identically distributed (non-IID) data, heterogeneous system, and malicious attacks.

In FL, each client trains a local model on its private dataset and submits local gradients to a Parameter Server (PS), who then aggregates these local gradients to update a global model. This process will keep on for multiple training rounds until the global model converges, in which the model aggregation plays an important role. The heterogeneous edge environment results in non-IID data because edge devices collect data based on their specific functions. Additionally, system heterogeneity arises from the varying computation and communication capabilities of different edge devices. The existence of non-IID data can hinder model convergence and negatively impact the accuracy of the trained models [2], [3], [4], particularly when using the FedAvg-like aggregation algorithms [1]. Meanwhile, dropping out devices or communication link failures will directly impact the model aggregation [5], [6]. To ensure model convergence in a heterogeneous environment, it is essential to develop a robust aggregation strategy that can adapt to non-IID data and heterogeneous systems.

When designing a robust aggregation strategy, it is crucial to ensure privacy protection. Local gradients can be vulnerable to inference attacks during the aggregation process [7], [8]. To address this concern, secure aggregation methods have been proposed to prevent the PS from inferring the privacy of local gradients [6]. However, most SA protocols go straight to aggregation algorithms that do not adapt to non-IID data.

Received 28 May 2024; revised 6 January 2025; accepted 28 March 2025. Date of publication 9 April 2025; date of current version 29 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 61802217, Grant 62372149, Grant 62372044, Grant U23A20303, and Grant U24B20146; in part by the Natural Science Foundation of Shandong Province under Grant ZR2023MF082; in part by the Key Laboratory of Knowledge Engineering with Big Data (Ministry of Education of China) under Grant BigKEOpen2025-04; in part by Qingdao Science and Technology Plan Key Research and Development Project under Grant 22-3-4-xxgg-10-gx; in part by the Original Exploration Project of Qingdao NSF under Grant 23-2-1-164-zyyd-jch; and in part by China Scholarship Council (CSC). The associate editor coordinating the review of this article and approving it for publication was Prof. Edgar Weippl. (Corresponding authors: Lingling Wang; Keke Gai.)

Lingling Wang, Mei Huang, Zhengyin Zhang, and Jingjing Wang are with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, Shandong 266061, China (e-mail: wanglingling@qust.edu.cn; huangmei@mails.qust.edu.cn; ZhangZhengyin@mails.qust.edu.cn; wangjingjing@qust.edu.cn).

Meng Li is with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, Anhui 230601, China (e-mail: mengli@hfut.edu.cn).

Keke Gai is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: gaikeke@bit.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2025.3559411>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3559411

Recent research [3], [9], [10] has indicated that the impact on the model convergence caused by non-IID data can be resolved by updating the aggregation weights¹ of clients in each training round. Nevertheless, existing weight updating algorithms [2], [11], [12] fail to meet privacy-preserving requirements, as clients send plaintext gradients or local data amounts to the PS responsible for updating the aggregation weights. As a result, the PS can deduce clients' private information from the aggregation weights, since both the data amount and gradient direction provide valuable insights about the clients' data [13]. Unfortunately, few studies address the need for privacy protection concerning the aggregation weights in SA.

Another factor that influences the aggregation strategy is the architecture of federated learning. In a typical Client/Server (C/S) architecture, the communication between the PS and the clients tends to be slow and unreliable. Additionally, there is a significant traffic overhead due to the frequent transmission of machine learning models between the PS and clients, which negatively impacts the model convergence rate. To solve these problems, an edge-assisted Hierarchical FL (HFL) architecture has been proposed in recent studies [14]. Specifically, clients send their local gradients to nearby edge servers (ESs) for aggregation. The ESs then send their intermediate aggregation results to the PS for global aggregation. This approach allows the ESs to alleviate the overwhelming workload on the PS and reduce the impact of slow communication links. Nevertheless, since ESs are generally not managed in-house like cloud servers [15], they are more likely to be compromised [16] in the open environment and submit incorrect aggregation results to undermine global model convergence.

These observations motivate us to study "how to guarantee robust and adaptive secure aggregation in an edge-assisted hierarchical FL with non-IID data, heterogeneous system, and malicious edge servers?" Though promising, it is still a non-trivial task and involves the following two challenges.

Challenge 1: It is challenging for existing SA protocols to achieve model convergence and improve the generalization ability of trained models in non-IID scenarios. On one hand, the straightforward method for accelerating model convergence is to select gradients similar to the global model and ignore heterogeneous gradients. But this decreases the model's generalization ability among heterogeneous clients. Therefore, more diverse gradients should be aggregated to maximize the model generalization ability. Nevertheless, the presence of more diverse gradients in the aggregation process leads to slower model convergence. On the other hand, SA must ensure the privacy of gradients during model aggregation. If the weight updates are carried out by a semi-honest PS, it could lead to privacy leaks because the aggregation weight is often determined by local gradients or the volume of data samples from clients.

Challenge 2: It is a challenge to address the system heterogeneity issue in the presence of malicious ESs. The heterogeneous system leads to stragglers and unreliable communication links. Current coding-based SA protocols for

addressing these problems exhibit low decoding efficiency and fail to withstand malicious attacks. Semi-honest straggling ESs affect model aggregation but not correctness. In contrast, a malicious ES can send arbitrary values to the PS instead of properly aggregating the clients' gradients, directly disrupting accurate model aggregation. This interference results in the unavailability of the global aggregation result and the training task has to start from scratch. Moreover, malicious ESs bring more challenges to the coding-based solutions in the heterogeneous system since it will directly affect the recovery threshold of the decoding process.

To solve the above challenges, we present RaSA, a robust and adaptive secure aggregation protocol by employing specifically designed techniques. We first design a novel adaptive aggregation strategy to address the non-IID data issue by considering the contribution of both gradient similarity and gradient diversity to the global model. The strategy works well when combined with one-time masking and secret sharing to ensure SA that prevent privacy leakage from local gradients and aggregation weights. In particular, the weight updates are calculated by local clients rather than edge servers or the PS, which eradicates the risk of weight privacy leakage. To address the system heterogeneity under straggling and malicious ESs, we construct verifiable coded computing [17] by introducing a verification key to linear codes and adopt Secret-shared Non-Interactive Proofs (SNIP) [18] to identify the malicious ESs. We take advantage of product codes followed by repetition coding to improve the decoding efficiency, where we regard the secure aggregation as a product-coded matrix multiplication of local gradients and aggregation weights at the ES side.

To summarize, the main contributions are as follows:

- We propose a robust and adaptive secure aggregation protocol that simultaneously addresses the non-IID data, system heterogeneity, and malicious attack issues in an edge-assisted HFL architecture. It guarantees the convergence rate and generalization ability of the trained model even when handling non-IID data and heterogeneous system while receiving a reliable global model resilient to malicious attacks launched by ESs.
- We significantly enhance the privacy of the protocol by protecting the gradient, the aggregation weight, and the weight updating process. Moreover, we improve the decoding efficiency of the global model update via the iterative decoding characteristics of product codes and the redundancy of repetition coding.
- We give a comprehensive theoretical analysis to prove the correctness and security of RaSA. Extensive experiments conducted on real-world datasets demonstrate that RaSA achieves high performance in terms of computational cost, accelerates the convergence speed, and also keeps the model generalization ability. Moreover, the decoding efficiency of RaSA achieves $1.33\times$ and $6.4\times$ faster than the state-of-the-art product-coded and one-dimensional coded computing schemes.

The remainder of this paper is organized as follows. Some related works are recapped in Section II. Section III presents the system model, threat model and assumptions, and design goals. Some preliminaries are in Section IV. Section V

¹The aggregation weight is assigned to each local gradient in the aggregation algorithm, which is different from the model's weights generated by the model training.

presents the design details of RaSA. Theoretical analysis and performance evaluation are presented in Sections VI and VII. Finally, Section VIII draws the conclusion.

II. RELATED WORK

In this section, we recap related works in three categories, i.e., secure aggregation with non-IID data, heterogeneous system, and malicious attacks.

A. Secure Aggregation With Non-IID Data

Recent works have explored the convergence behavior of FedAvg in realistic federated settings with non-IID data [19]. Research results show that non-IID data slows down the convergence speed when using FedAvg-like aggregation algorithms [2], [3], [4]. Some studies [10], [11], [20], [21], [22] have made steps towards convergence guarantees by devising weighted aggregation algorithms, where the aggregation weights of each iteration are updated via different weight updating algorithms based on the number of training data, the speed of local training, the client contribution, or the similarity between the local gradient and the global gradient. Rodio et al. [2] proposed to assign different aggregation weights to each client by solving an optimization problem that achieves a good trade-off in heterogeneous FL settings. Wang et al. [9] presented Adaptive-Krum for the PS to aggregate gradients, which exploited the contribution of the model diversity to the model convergence rather than the model similarity. Existing aggregation strategies focus on mitigating the negative impact of non-IID data on test accuracy and model convergence [23]. However, their weight calculations are all performed by the PS via plaintext gradients, which violates the privacy protection of SA. By contrast, RaSA ensures the privacy of both aggregation weights and local gradients. Specifically, the clients calculate the weights themselves, which ensures that the PS does not gain access to plaintext gradients or plaintext weights.

B. Secure Aggregation With Heterogeneous System

To address the heterogeneous system, some early work [24] proposed to select high-quality clients, which helps mitigate the impact of resource-limited clients or limited network bandwidth on the aggregation process. The selected clients are required to keep online for the entire lifetime of the FL task. Hence, the stragglers or dropping out clients will impede the model aggregation. Considering the secure aggregation under heterogeneous systems, some protocols [6], [25], [26], [27], [28] have utilized one-time pad and secret sharing to enable the reconstruction of missing values and subsequent cancellation of delays caused by stragglers. However, these protocols require frequent interaction between clients and server, resulting in a huge communication overhead which increases with the clients' dropout rate. To address this issue, recent studies proposed to adopt coding theory to enhance stragglers' resiliency [5], [29], [30], [31]. Most coding-based studies [32], [33], [34], [35], [36] adopt Maximum Distance Separable (MDS) codes to provide straggler resiliency in heterogeneous FL system. Unfortunately, most studies suffer from

decoding inefficiency since the gradient is a high-dimensional vector. In RaSA, we combine product codes with repetition codes to improve the decoding efficiency.

C. Secure Aggregation With Malicious Attacks

In a C/S architecture, malicious attacks will be launched in the SA process by untrusted clients. To resist the influence of malicious clients on model aggregation, recent studies extended SA with privacy-preserving input validation by using norm boundary detection [37], zero-knowledge proofs [38], [39], [40], etc. In an edge-assisted HFL architecture, existing SA protocols have focused on addressing dropouts and privacy issues in a semi-honest model [41], [42]. However, since edge servers are more likely to be compromised in the open environment, they may violate the privacy and integrity of the model aggregation. Thus, verifying the behavior of the ESs is essential to eliminate the impact of malicious attacks on the availability of the aggregation result. Unfortunately, few studies focus on malicious attacks from the ESs in HFL.

Unlike prior works, we investigate malicious attacks of edge servers during partial model aggregation in an edge-assisted HFL and design a verifiable approach to verify the correctness of the aggregation result. Moreover, we are the first to simultaneously address the non-IID data, system heterogeneity and malicious attack issues in SA. This task is complex and not trivial. Since these issues are interconnected, we must analyze their relationships, develop new aggregation strategies, and create suitable technologies to support our protocol. Table I compares RaSA with prior works.

III. PROBLEM STATEMENT

In this section, we formalize the system model, threat model and assumptions, and present our design goals. The notations used in this paper are shown in Table II.

A. System Model

The system model mainly consists of a parameter server, several edge servers, and multiple clients as shown in Fig. 1.

Parameter Server (PS) is responsible for publishing public parameters. The PS also aggregates partial aggregation results, decodes and verifies the global aggregation result, aggregates weight updating parameters, and maintains a malicious set.

Edge Servers (ESs) $\{ES_1, ES_2, \dots, ES_e\}$ provide unstable connections to the PS and have sufficient resources in computation. They receive the secret shares of the masked aggregation weights and local gradients from clients, perform partial aggregation, generate integrity proofs of partial aggregation, and verify the integrity proofs from other ESs if needed.

Clients $\{P_1, P_2, \dots, P_u\}$ are heterogeneous devices with various degrees of reliability and can stop responding due to device or network failure. They use their private data to get local gradients, mask and encode local gradients, and then submit them to different edge servers. Moreover, they are also responsible for updating their aggregation weights locally and sharing the encoded aggregation weights secretly with the ESs.

TABLE I
COMPARISON OF FUNCTIONALITY WITH THE MOST RELEVANT WORKS

Scheme	Architecture	Threat Model	Robustness			Privacy Protection	
			Non-IID Data	Heterogeneous System	Malicious Attacks	Gradients	Aggregation Weights
Liao et al [4]	Single server	honest	✓	✓	×	×	×
FLTrust [11]	Single server	malicious	✓	×	✓	×	×
FedEdge [9]	Edge-Assisted	malicious	✓	×	✓	×	×
LCM [36]	Multi-Server	semi-honest	×	✓	×	✓	×
RaSA (Ours)	Edge-Assisted	malicious	✓	✓	✓	✓	✓

TABLE II
KEY NOTATIONS

Notation	Definition
θ^t	the global model of round t
θ_i^t	the local model of client P_i at round t
$\mathbb{F}(q)$	Finite field of order q
$\mathbf{G}$	Generator matrix of LRC codes
$\mathbf{I}$	Allocation matrix
e	Number of edge servers
u	Number of clients
k	Number of original data fragments
n	Number of coding blocks of LRC codes
m	Maximum number of malicious edge servers
$(\cdot)^T$	Transpose of vectors and matrices
g_i	Local gradient of client P_i
$\mathbf{g}^t$	Global gradient of round t
w_i	Aggregation weight of client P_i
ρ_1	Redundancy rate of (n, k) -LRC code
ρ_2	Repetition degree
$\tilde{\mathbf{M}}$	Masked gradients of u clients
$\tilde{\mathbf{W}}$	Masked aggregation weights of u clients
$\mathbf{X}_{:,i}$	the i -th column of the matrix $\mathbf{X}$
$\mathbf{X}_{i,:}$	the i -th row of the matrix $\mathbf{X}$
$\mathbf{C}$	Coded gradient matrix of clients
$\mathbf{S}$	Coded weight matrix of clients
b_i, v_i	Random seed for $\text{PRG}(\cdot)$
R_i, V_i	Random number from $\text{PRG}(b_i), \text{PRG}(v_i)$
γ	Boundary angle
η	Decoding threshold of the product codes
$\mathbf{D}_i$	Local training data of client P_i
$ \mathbf{D}_i $	Number of samples in $\mathbf{D}_i$
μ	Fractional cache size
$\varepsilon_{i,j}$	Masking number of gradient $g_{i,j}$
$\epsilon_{i,j}$	Masking number of aggregation weight $w_{i,j}$
$\alpha_i(t)$	Cosine similarity between g_i and θ^t

The workflow of RaSA is as follows: ① The PS publishes the parameters of an FL task. ② Clients perform the local training to obtain local gradients, update aggregation weights, mask gradients and aggregation weights, and encode them. ③ Clients submit secret shares of the encoded local gradients and aggregation weights to different ESs. ④ The ESs receive secret shares from clients and perform partial aggregation, generating interactive integrity proofs for other ESs. ⑤ ESs upload the partial aggregation results to the PS. ⑥ The PS calculates public parameters needed for the weights updating. Subsequently, the PS aggregates partial aggregation results from at least η ($\eta \leq e$) edge servers and decodes the global aggregation result, and then verifies its correctness. If it is correct, compute the global model for the next training round. Otherwise, ⑦ to detect the malicious edge servers, edge servers partake in the verification of the proofs from other ESs under the supervision of the PS. ⑧ The PS discards the malicious partial aggregation results and reconstructs the correct global model for the next training round.

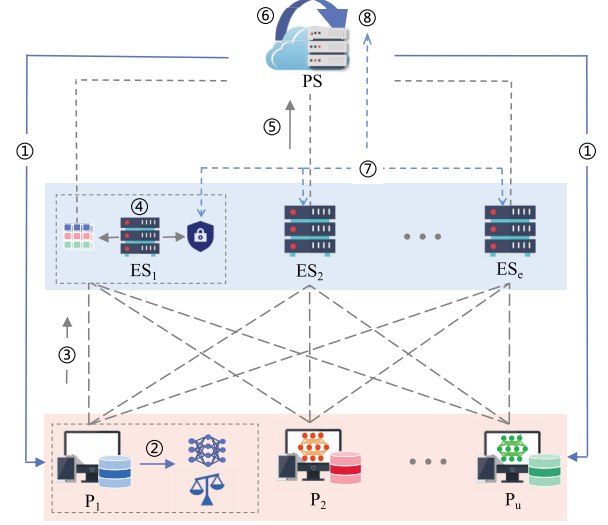


Fig. 1. System model.

B. Threat Model and Assumptions

We assume that the PS and clients are semi-honest, i.e., they follow the protocol faithfully, but are curious about the sensitive data of the involved clients. They passively launch privacy attacks to obtain clients' privacy. Edge servers are untrusted in our model, and they may deviate from the procedure of partial aggregation or be maliciously manipulated during the FL task. Specifically, they may 1) return arbitrary values as the partial aggregation results without executing the whole procedure of the partial aggregation; and 2) modify the partial aggregation results. They may also launch collusive attacks on the clients' local gradients to obtain private information. However, the edge servers will not collude with the PS.

We assume there are at most $m < e - \eta$ malicious ESs, and treat malicious ES as a straggler. Assume that a random subset of clients or upper bound by $e - \eta$ edge servers may drop out at any point in time since they disconnect or are too slow to respond due to unstable network connectivity, power loss, etc. Since the security and privacy attacks in FL are mainly from internal adversaries, we focus on the impact of internal attackers on the protocol.

C. Design Goals

We aim to achieve a robust and privacy-enhanced adaptive aggregation protocol in HFL with an open and heterogeneous edge environment.

Robustness: RaSA provides robustness in the presence of non-IID data, system heterogeneity, and malicious edge

servers. Firstly, RaSA should guarantee the goals of accelerating model convergence speed and enhancing model generalization ability under the non-IID data scenario. Secondly, RaSA should guarantee the correct model aggregation even if there are a subset of clients or edge servers that are temporarily offline at any time, or some communication links are faulty. Thirdly, RaSA should guarantee that malicious edge servers can not interfere with model aggregation via model poisoning attacks. In particular, the PS should be able to verify the correctness of partial aggregation results, reject incorrect results, and calculate the correct global model without having to train from scratch.

Privacy Protection: RaSA protects local gradients and aggregation weights against the colluding edge servers and the semi-honest PS. Specifically, any subset of up to $\mathcal{C}$ colluding edge servers learn nothing about clients' local gradients and aggregation weights. The PS learns nothing about the local gradients and aggregation weights from the partial aggregation results, weight updating parameters, and proof summaries.

IV. PRELIMINARIES

A. Product-Coded Computing

Coded computing [43] has been used to tolerate stragglers through linear coding of rows of the matrix in distributed computing systems. It can speed up the distributed matrix multiplication compared to uncoded ones. Coded computing schemes usually encode computation along one dimension of the matrix by using MDS codes. However, they are only suitable for the case where one dimension of the matrix is small enough. For higher-dimensional linear operations such as tensor operations, the product-coded computing [44] is proposed to encode computation along both dimensions, which can achieve a better decoding efficiency than one dimensional coded scheme. In RaSA, we employ product-coded computing to reduce the decoding latency of the global aggregation result. To further improve the decoding efficiency, we use a Locally Recoverable Code (LRC) [45] to encode the masked gradients and aggregation weights of the two dimensions that make up the matrix, which enables to repair missing encoded data via a small amount of data instead of all information contained in a codeword.

B. Pseudorandom Generators and Functions

A Pseudorandom Generator (PRG) [46] is a deterministic procedure that produces a pseudorandom distribution from a uniformly chosen seed, which guarantees that the output distribution on the seed is computationally indistinguishable from a uniform distribution. In RaSA, we require a $PRG(\cdot)$ with a random seed $\{0, 1\}^\lambda$ to expand it into a longer string, where λ is the security parameter. The output of the PRG is viewed as a random number in $GF(q)$ instead of a binary string. We use the pseudorandom number generated by $PRG(\cdot)$ to mask clients' weight updating parameters and aggregation weights. Moreover, we adopt a pseudorandom function $PRF(\cdot)$ [47], which is a family of deterministic functions indexed by a key that maps a chosen seed to a new seed that the indexed function is computationally indistinguishable from a

truly random function. $PRF(\cdot)$ is used to generate different random seeds in each round, which ensures the security of RaSA over multiple training rounds and removes the additional interactions before each round.

C. Threshold Secret Sharing With Repetition Coding

Threshold Secret Sharing [48] is a cryptographic primitive for taking a secret, breaking it up into multiple shares, and distributing the shares among multiple parties (each party is assigned a single share), so that only when the authorized subset of parties (no less than recovery threshold) bring together their respective shares can the secret be reconstructed. If the number of shares is less than the recovery threshold, the secret cannot be recovered. This creates a challenge for edge scenarios where communication link failures and dropping ESs exceed the recovery threshold. To enhance the ability to withstand dropouts, threshold secret sharing can be combined with repetition coding [49]. In this approach, each participant is assigned multiple shares instead of just one. This method can reduce the number of parties required to recover the secret. Research has shown that the redundancy introduced by repetition coding can be utilized to improve resilience against dropouts [50]. For brevity, we use "repetition-based secret sharing" to represent the threshold secret sharing with repetition coding. To improve the decoding efficiency, RaSA adopts product-coded computing along with repetition-based secret sharing to address the system heterogeneity issue.

D. Secret-Shared Non-Interactive Proof

The Secret-sharing Non-Interactive Proof (SNIP) [18] is an information-theoretic zero-knowledge proof, which is designed for a multi-verifier setting where the private data is distributed or secret-shared among the verifiers. All parties first share a public arithmetic circuit representing a validation predicate $Valid(\cdot): GF(q)^d \rightarrow GF(q)$, and the prover holds private information $x \in GF(q)^d$, where q is a prime number and $GF(q)$ is chosen such that $2M \ll |GF(q)|$, and M is the number of multiplication gates in $Valid(\cdot)$. In RaSA, we adopt SNIP to identify the malicious ES that violates the integrity of the partial aggregation process.

V. DESIGN DETAILS

In this section, we first provide a technical overview of how we address the two challenges outlined in Section I. Then we present the detailed protocol.

A. Privacy-Preserving Weight Updating to Address Non-IID Data Issue in SA

We design a privacy-preserving weight updating strategy to address the privacy leakage and non-IID data issues in SA. To solve the challenges posed by non-IID data on model convergence and generalization, we propose an adaptive weight updating algorithm, as shown in Algorithm 1. This algorithm is executed by local clients rather than edge servers or the PS, which helps reduce the risk of privacy leakage. Moreover, we utilize pseudorandom numbers generated by a PRG and PRF to mask weight updating parameters because the PS needs to

Algorithm 1 Adaptive Weight Updating

Input : Global gradient $\mathbf{g}^{t-1}$, number of samples $|\mathbf{D}_i|$, local gradient $\mathbf{g}_i^t$ at round t , weight updating parameters $\sum_{i=1}^u \widetilde{WF}_i^{t-1}$, $\sum_{i=1}^u \widetilde{\alpha}_i(t-1)$, random seeds $\{v_1, v_2, \dots, v_u\}$.

Output : Aggregation weight w_i^t , weight updating factor WF_i^t , smoothed angle $\widetilde{\alpha}_i(t)$.

```

1 for Round  $t = 1, 2, \dots$  do
2   if  $t = 1$  then
3     The smoothed angle is  $\widetilde{\alpha}_i(t) = \frac{\pi}{4}$ ;
4     The boundary angle is  $\gamma^t = \frac{\pi}{4}$ ;
5   else
6     Calculate the angle between  $\mathbf{g}_i^t$  and  $\mathbf{g}^{t-1}$ :
7      $\alpha_i(t) = \arccos \frac{\langle \mathbf{g}^{t-1}, \mathbf{g}_i^t \rangle}{\|\mathbf{g}^{t-1}\| \|\mathbf{g}_i^t\|}$ ;
8     Compute  $\widetilde{\alpha}_i(t) = \frac{t-1}{t} \widetilde{\alpha}_i(t-1) + \frac{1}{t} \alpha_i(t)$ ;
9     Update random seeds  $v_j' = PRF(v_j, t-1)$ ,
10     $j \in [u]$ ;
11    Compute the sum of pseudorandom numbers
12     $\sum_{j=1}^u V_j^{t-1} = \sum_{j=1}^u PRG(v_j')$ ;
13    Compute weight updating factor
14     $\sum_{j=1}^u WF_j^{t-1} = \sum_{j=1}^u \widetilde{WF}_j^{t-1} - \sum_{j=1}^u V_j^{t-1}$ ;
15    Compute the boundary angle
16     $\gamma^t = \frac{1}{u} (\sum_{j=1}^u \widetilde{\alpha}_j(t-1) - \sum_{j=1}^u V_j^{t-1})$ ;
17  if  $0 < \widetilde{\alpha}_i(t) \leq \gamma^t$  then
18    Calculate  $WF_i^t = |\mathbf{D}_i| e^{\cos(\widetilde{\alpha}_i(t)) \|\mathbf{g}_i^t\|}$ ;
19  else if  $\gamma^t < \widetilde{\alpha}_i(t) < \frac{\pi}{2}$  then
20     $WF_i^t = |\mathbf{D}_i| e^{\sin(\widetilde{\alpha}_i(t) - \gamma^t) \|\mathbf{g}_i^t\|}$ ;
21  Return  $w_i^t = \frac{WF_i^t}{\sum_{j=1}^u WF_j^{t-1}}$ ,  $WF_i^t$ ,  $\widetilde{\alpha}_i(t)$ .
```

aggregate these parameters to ensure adaptive weight updating. To improve model generalization ability, the algorithm combines the contributions of both gradient similarity and gradient diversity. To enhance the model convergence rate, aggregation weights are updated according to the non-IID degree of local data, which are calculated from two dimensions of diversity: gradient direction and the number of data samples.

Gradient direction. The direction of the local gradient usually points to the distribution of the client's local datasets. Compared to the global gradient, if the client's gradient is far from the direction of the global gradient, it means that the client has much more diverse samples than other clients. Existing aggregation schemes prefer to either only similar gradients, i.e., those of which the direction is closer to the global gradient [11], [51], or only diverse gradients that are far from the global gradient [9]. Hence, it is difficult to guarantee model convergence speed and enhance model generalization ability at the same time in non-IID scenarios.

Different from previous works, we calculate cosine similarity to measure the angle $\alpha(t)$ in the direction between the local gradient $\mathbf{g}_i$ and global gradient $\mathbf{g}^t$. We define a local gradient with a small angle as a similar gradient and one with a large angle as a heterogeneous gradient. To distinguish

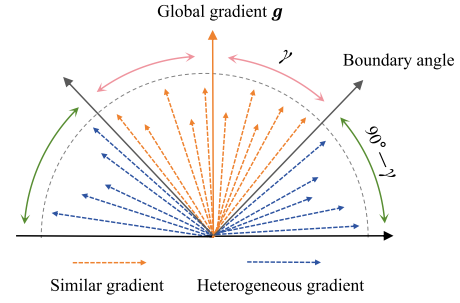


Fig. 2. Similar and heterogeneous gradients divided by boundary angle.

them, we set a boundary value of the angle γ dynamically, relative to the direction of the global gradient. For instance, it can be the average of all clients' angles of each training round. In Fig. 2, when $0 < \alpha(t) < \gamma$ and $\alpha(t)$ approaches to 0, it indicates that the local gradient is more similar to the global gradient and thus makes more contributions to the global gradient. Consequently, it should be assigned a higher aggregation weight. When $\gamma < \alpha(t) < 90^\circ$, as $\alpha(t)$ gets closer to 90° , it indicates that the client has more heterogeneous samples than other clients, and the training accuracy on these samples surpasses that of other users. Hence, the aggregation weights should also be boosted rather than reduced. However, in general, local gradients that are more similar to the global gradient contribute more to the model convergence, so the aggregation weights of clients with the $\alpha(t)$ in the range $(0, \gamma]$ should be higher than those in the range $(\gamma, 90^\circ)$.

Number of samples. Non-IID degree is also reflected in the number of data samples. It is generally believed that clients with more training samples contribute more to the global model [19]. Hence, the aggregation weight should also be positively correlated with the number of samples $|\mathbf{D}|$. We define a weight updating factor WF to reflect the impact of different non-IID degrees on the weight updates. The WF should be computed adaptively based on the angle $\alpha(t)$, γ and $|\mathbf{D}|$. To restrain the instability caused by randomness presented in instantaneous $\alpha(t)$ at each round, we adopt the smoothed angle $\widetilde{\alpha}(t)$, which is the average angle of previous training rounds. By using the smoothed angle $\widetilde{\alpha}(t)$, the angle difference among clients uniquely depends on the data distribution. The details can be found in Section V-C.

B. Verifiable Coded Computing to Address the System Heterogeneity Issue in the Presence of Malicious ESs

We utilize verifiable coded computing to address the system heterogeneity issue in a hybrid aggregation with malicious ESs, including the partial aggregation at the ES side and the global aggregation at the PS side. Since straggling clients and unstable communication links between clients and ESs will not affect the correctness of model aggregation, RaSA focuses on addressing the system heterogeneity issue caused by straggling ESs and unstable communication links between the PS and ESs. Technically, we construct verifiable coded computing [17] by introducing a verification key to linear codes, which helps eliminate the influence of malicious ESs on the aggregation process. We take advantage of the consistency of the random

number shared between the PS and clients to verify the correctness of the global aggregation result. In cases where the global aggregation result is incorrect, malicious ESs can be identified using a modified version of SNIP [38]. The correct global aggregation result can then be recalculated by excluding the incorrect partial results, avoiding the need to retrain from scratch. Moreover, we employ product codes followed by a repetition code to improve the decoding efficiency, where we regard the secure partial aggregation as a product-coded matrix multiplication [44] of local gradients and aggregation weights at the ES side. The redundancy of repetition codes helps lower the decoding latency at the PS side.

Specifically, RaSA allows u clients, a single PS, and e ESs with a maximum of C colluding ESs and m malicious ESs to collaboratively train a global model securely. For decoding efficiency, we adopt (n, k) -LRC code to encode the masked gradients $\tilde{\mathbf{M}} \in GF(q)^{u \times k}$ and aggregation weights $\tilde{\mathbf{W}} \in GF(q)^{k \times u}$, the product of which refers to the masked global aggregation result. Partial aggregation process at different ES side only produces a subset of these product-coded results. In the case of dropped ESs, the PS needs to get enough intermediate results to restore the global aggregation result. To reduce the decoding latency at the PS side, a repetition-based secret sharing is adopted so that each client transmits $\beta + 1$ ($\beta < n-1$) coding blocks to the corresponding ESs according to an allocation matrix $\mathbf{I} \in GF(q)^{(\beta+1) \times e}$ which is designed in a cyclic manner. Considering the storage constraint, each ES can keep an equivalent to a fraction $\mu \in [\frac{1}{e}, 1)$ of $\tilde{\mathbf{W}}$ and $\tilde{\mathbf{M}}$ for the sake of privacy consideration. For the recovery requirement, at least η partial aggregation results are needed. The settings of these parameters are described in Theorem 4.

Next, we will explain how to construct the product codes in RaSA. The gradients and the aggregation weights are firstly divided into $k-1$ shares and then masked by a random vector to form a k -dimensional vector for privacy. Then, the masked gradients of u clients can form a matrix $\tilde{\mathbf{M}} \in GF(q)^{u \times k}$, and each row of the matrix $\tilde{\mathbf{M}}$ corresponds to a masked gradient with dimension k . In the same way, the masked aggregation weights of u clients can form a matrix $\tilde{\mathbf{W}} \in GF(q)^{k \times u}$, and each column of the matrix $\tilde{\mathbf{W}}$ corresponds to a k -dimensional masked aggregation weights. To generate the product codes of $\tilde{\mathbf{W}} \times \tilde{\mathbf{M}}$, every row of the $\tilde{\mathbf{M}}$ is encoded, resulting in a matrix $\mathbf{C} \in GF(q)^{u \times n}$. Similarly, every column of the $\tilde{\mathbf{W}}$ is encoded to get a matrix $\mathbf{S} \in GF(q)^{n \times u}$. After that, the product codes $\mathbf{S} \times \mathbf{C} \in GF(q)^{n \times n}$ can be obtained, which includes n^2 product-coded results $\mathbf{S}^{(i)} \times \mathbf{C}^{(j)}$, where $i, j \in [n]$. As long as k or more results are collected from any row or column, the entire row or column can be reconstructed via the decoding algorithm of the underlying LRC codes. Decoding can be done iteratively between row and column decoders until each row and column is completely reconstructed, even if the missing blocks exceed $n-k$.

To keep track of malicious ESs, the PS maintains a set S_m^* containing the list of malicious ESs. When the global aggregation result fails to pass the verification via the random numbers, the ES_i needs to participate in the verification of the proofs from all other edge servers $ES_j \in ES \setminus i$ via SNIP. Through this process, the ES_j that has not correctly executed the local aggregation process will be identified and flagged as

malicious by the PS, i.e., $S_m^* \leftarrow S_m^* \cup ES_i$. Finally, the PS only aggregates the correct partial aggregation results via discarding the incorrect results according to S_m^* . In RaSA, we make some modifications for SNIP to support our protocol, where the secret shares of integrity proofs need not be encrypted since the inputs for $Valid(\cdot)$ are coded gradient shares and coded weight shares. Even if malicious ESs collude to obtain C inputs of $Valid(\cdot)$, they can only reconstruct the global model through decoding and are unable to access any privacy from the encoded shares, which does not compromise the security of RaSA. By contrast, SNIP [38] needs to encrypt secret shares of integrity proofs to protect the privacy of input (local gradient) for the circuit $Valid(\cdot)$. Due to space constraints, we describe the process of generating and verifying integrity proofs for the modified SNIP in our submitted *Supplemental Materials*.

C. Detailed Protocol

We present the detailed procedure of RaSA, including system initialization, local operation, partial aggregation, and global aggregation, as shown in Protocol 2. The workflow of RaSA is also demonstrated in Fig. 3. Specifically, the PS first distributes an FL task and publishes public parameters $\{\theta^0, Valid(\cdot), GF(q), PRG(\cdot), PRF(\cdot), \mathbf{G}_{k \times n}, \mathbf{I}_{(\beta+1) \times e}, \gamma^1, w^1\}$, where $\mathbf{G}_{k \times n}$ is the generator matrix, $\mathbf{I}_{(\beta+1) \times e}$ is the allocation matrix generated by a cyclic permutation group of order e with generator $\pi = \{1, 2, \dots, n\}$.

$$\mathbf{I} = \begin{pmatrix} \pi^0(1) & \pi^0(2) & \cdots & \pi^0(e) \\ \pi^{e-p}(1) & \pi^{e-p}(2) & \cdots & \pi^{e-p}(e) \\ \vdots & \vdots & \ddots & \vdots \\ \pi^{\beta(e-p)}(1) & \pi^{\beta(e-p)}(2) & \cdots & \pi^{\beta(e-p)}(e) \end{pmatrix}$$

The clients obtain local gradients through local model training and the aggregation weights are updated via Algorithm 1. To prepare for encoding, they first map the real-domain gradients and aggregation weights onto the finite field [52]. Next, the clients mask their local gradients and aggregation weights using pseudorandom numbers, encode the masked gradients and aggregation weights via LRC codes, and then secretly share the coding blocks with edge servers according to the allocation matrix $\mathbf{I}_{(\beta+1) \times e}$. After receiving the coding blocks from the clients, the ESs perform partial aggregation and generate the integrity proofs via SNIP. The PS aggregates partial aggregation results, decodes them to obtain the masked global aggregation result

$$\tilde{\mathbf{W}} \times \tilde{\mathbf{M}} = \begin{bmatrix} \tilde{\mathbf{W}}_{1,:} \tilde{\mathbf{M}}_{:,1} & \tilde{\mathbf{W}}_{1,:} \tilde{\mathbf{M}}_{:,2} & \cdots & \tilde{\mathbf{W}}_{1,:} \tilde{\mathbf{M}}_{:,k-1} & \tilde{\mathbf{W}}_{1,:} \\ \tilde{\mathbf{W}}_{2,:} \tilde{\mathbf{M}}_{:,1} & \tilde{\mathbf{W}}_{2,:} \tilde{\mathbf{M}}_{:,2} & \cdots & \tilde{\mathbf{W}}_{2,:} \tilde{\mathbf{M}}_{:,k-1} & \tilde{\mathbf{W}}_{2,:} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{R}^t \tilde{\mathbf{M}}_{:,1} & \mathbf{R}^t \tilde{\mathbf{M}}_{:,2} & \cdots & \mathbf{R}^t \tilde{\mathbf{M}}_{:,k-1} & \mathbf{R}^t \end{bmatrix}$$

and verifies the correctness of $\tilde{\mathbf{W}} \times \tilde{\mathbf{M}}$ via the consistency of $\mathbf{R}^t$, where $\tilde{\mathbf{M}}_{:,i} = [\tilde{g}_{1,i}, \tilde{g}_{2,i}, \dots, \tilde{g}_{u,i}]^T$ and $\tilde{\mathbf{W}}_{i,:} = [\tilde{w}_{1,i}, \tilde{w}_{2,i}, \dots, \tilde{w}_{u,i}]$. If the masked global aggregation result fails to pass the verification equation, the PS notifies all ESs to verify the integrity proofs of all other ESs and flag the malicious ESs, and finally obtains the malicious set S_m^* via SNIP. In the end,

Protocol 1 RaSA: Robust and Adaptive Secure Aggregation for Edge-Assisted Hierarchical Federated Learning.**1 System Initialization:****2 Setup (PS):**

- 3 Distribute an FL task and publish the initial global model θ^0 and the circuit $Valid(\cdot)$;
- 4 Select a finite field $GF(q)$ of prime order $q > 2^\lambda$, where $\lambda \ll n$ is the secure parameter, a number $p \in \mathbb{N}$ such that $p/e \leq \mu$ and $\mu \in [\frac{1}{e}, 1)$, a pseudorandom generator $PRG(\cdot): \{0, 1\}^\lambda \rightarrow GF(q)$, and a pseudorandom function $PRF(\cdot): \kappa \times \chi \rightarrow \Upsilon$;
- 5 Construct an invertible matrix $\mathbf{G}_{k \times n}$ as the generator matrix for (n, k) -LRC codes;
- 6 Generate the allocation matrix $\mathbf{I}_{(\beta+1) \times e}$ by determining parameters $\{\rho_1, \rho_2\}$, where $\beta = \lceil \rho_1 \rho_2 k / e - 1 \rceil$;
- 7 Initialize a malicious set $S_m^* = \emptyset$ and a malicious table $\mathcal{M}_{e \times e}^{tab} = null$;
- 8 Publish the initial aggregation weight $w^1 = \frac{1}{u}$, boundary angle $\gamma^1 = \frac{\pi}{4}$, and smoothed angle $\tilde{\alpha}_i(1) = \frac{\pi}{4}$;
- 9 Transmit random seeds $\{b_1, b_2, \dots, b_u\}$ secretly to clients $\{P_1, P_2, \dots, P_u\}$ respectively.
- 10 Client P_i selects a random seed v_i , sends it to other clients, and keeps v_i secret from the PS.

11 Local Operation (P_i at round t):**12 Weight updating:**

- 13 Receive weight updating parameters $\sum_{i=1}^u \widetilde{WF}_i^{t-1}$ and $\sum_{i=1}^u \widetilde{\alpha}_i(t-1)$ from the PS, where $t > 1$;
- 14 Obtain the aggregation weight w_i^t , weight updating factor WF_i^t and smoothed angle $\tilde{\alpha}_i(t)$ via Algorithm 1;
- 15 Generate random numbers $V_i^t = PRG(v_i^t)$, where $v_i^t = PRF(v_i, t)$;
- 16 Mask WF_i^t and $\tilde{\alpha}_i(t)$ via V_i^t : $\widetilde{WF}_i^t = WF_i^t + V_i^t$, $\widetilde{\alpha}_i(t) = \tilde{\alpha}_i(t) + V_i^t$;
- 17 Transmit $\widetilde{WF}_i^t$ and $\widetilde{\alpha}_i(t)$ to the PS secretly.

18 Masking:

- 19 Divide the local gradient g_i and the aggregation weight w_i into $k-1$ fragments, where $\sum_{j=1}^{k-1} g_{i,j} = g_i$ and $\sum_{j=1}^{k-1} w_{i,j} = w_i$;
- 20 Generate two random sequences $\{\varepsilon_{i,1}, \varepsilon_{i,2}, \dots, \varepsilon_{i,k-1}, 0\} \in GF(q)^k$ and $\{\epsilon_{i,1}, \epsilon_{i,2}, \dots, \epsilon_{i,k-1}\} \in GF(q)^{k-1}$, where $\sum_{j=1}^{k-1} \varepsilon_{i,j} = 0$ and $\sum_{j=1}^{k-1} \epsilon_{i,j} = 0$;
- 21 Generate random numbers $R_i^t = PRG(b_i^t)$, where $b_i^t = PRF(b_i, t)$;
- 22 Mask the local gradient g_i as $\tilde{g}_i = (g_{i,1}, g_{i,2}, \dots, g_{i,k-1}, 1) + (\varepsilon_{i,1}, \varepsilon_{i,2}, \dots, \varepsilon_{i,k-1}, 0) \rightarrow \tilde{M}_{i,:}$;
- 23 Mask the aggregation weight w_i as $\tilde{w}_i = (w_{i,1}, w_{i,2}, \dots, w_{i,k-1}, 0) + (\epsilon_{i,1}, \epsilon_{i,2}, \dots, \epsilon_{i,k-1}, R_i^t) \rightarrow \tilde{W}_{:,i}$;

24 Encoding:

- 25 Encode $\tilde{g}_i$ and $\tilde{w}_i$ via (n, k) -LRC codes as following:
- 26 $[\tilde{g}_{i,1} \ \tilde{g}_{i,2} \ \dots \ \tilde{g}_{i,k}] \times \mathbf{G}_{k \times n} = [C_i^{(1)} \ C_i^{(2)} \ \dots \ C_i^{(n)}] = C_{i,:}$;
- 27 $[\tilde{w}_{i,1} \ \tilde{w}_{i,2} \ \dots \ \tilde{w}_{i,k}] \times \mathbf{G}_{k \times n}^T = [S_i^{(1)} \ S_i^{(2)} \ \dots \ S_i^{(n)}]^T = S_{:,i}$;
- 28 Send $\beta + 1$ coding blocks of $C_{i,:}$ and $S_{:,i}$ to the corresponding ES according to the allocation matrix $\mathbf{I}$.

29 Partial Aggregation (ES_j at round t):

- 30 Receive $\beta + 1$ coding blocks of the local gradients and aggregation weights respectively, i.e., $[C^{(\pi^0(j))} \ C^{(\pi^{e-p}(j))} \ \dots \ C^{(\pi^{\beta(e-p)}(j))}]$ and $[S^{(\pi^0(j))} \ S^{(\pi^{e-p}(j))} \ \dots \ S^{(\pi^{\beta(e-p)}(j))}]^T$ based on the j_{th} column of $\mathbf{I}$ ($[\pi^0(j) \ \pi^{e-p}(j) \ \dots \ \pi^{\beta(e-p)}(j)]^T$) from all online clients;
- 31 Perform the partial aggregation: $[S^{(\pi^0(j))} \ S^{(\pi^{e-p}(j))} \ \dots \ S^{(\pi^{\beta(e-p)}(j))}]^T \times [C^{(\pi^0(j))} \ C^{(\pi^{e-p}(j))} \ \dots \ C^{(\pi^{\beta(e-p)}(j))}]$;
- 32 Generate integrity proof Π_{ji} and the check string $\Psi_{\Pi_{ji}}$ for other edge servers $ES_i \in ES_{\setminus j}$ via SNIP.

33 Global Aggregation (PS at round t):

- 34 Aggregate the weight updating factors $\sum_{i=1}^u \widetilde{WF}_i^t$ and smoothed angles $\sum_{i=1}^u \widetilde{\alpha}_i(t)$, and publish the results;
- 35 Aggregate the partial aggregation results to obtain $\mathbf{S} \times \mathbf{C} = [S^{(1)} \ S^{(2)} \ \dots \ S^{(n)}]^T \times [C^{(1)} \ C^{(2)} \ \dots \ C^{(n)}]$ and decode it to get the masked global aggregation result $\tilde{\mathbf{W}} \times \tilde{\mathbf{M}}$;
- 36 Compute the sum of random number $\mathbf{R}^t = \sum_{i=1}^u PRG(b_i^t)$, where $b_i^t = PRF(b_i, t)$;
- 37 **if** $(\tilde{\mathbf{W}} \times \tilde{\mathbf{M}})_{k,k} = \mathbf{R}^t$ **then**
- 38 Obtain the global aggregation result $\mathbf{W} \times \mathbf{M} = \sum_{i=1}^{k-1} \sum_{j=1}^{k-1} (\tilde{\mathbf{W}} \times \tilde{\mathbf{M}})_{i,j}$;
- 39 **else**
- 40 Notify ESs for interactive verification by using integrity proof Π and the check string Ψ via SNIP;
- 41 Treat malicious ESs as dropouts and aggregate correct partial aggregation results by looking up S_m^* ;
- 42 Decode to obtain $\mathbf{W} \times \mathbf{M}$;
- 43 Map the aggregation back to the real domain to obtain the global gradient $\mathbf{g}^t$ and broadcast it;
- 44 Compute the new global model θ^t and turn to the next round.

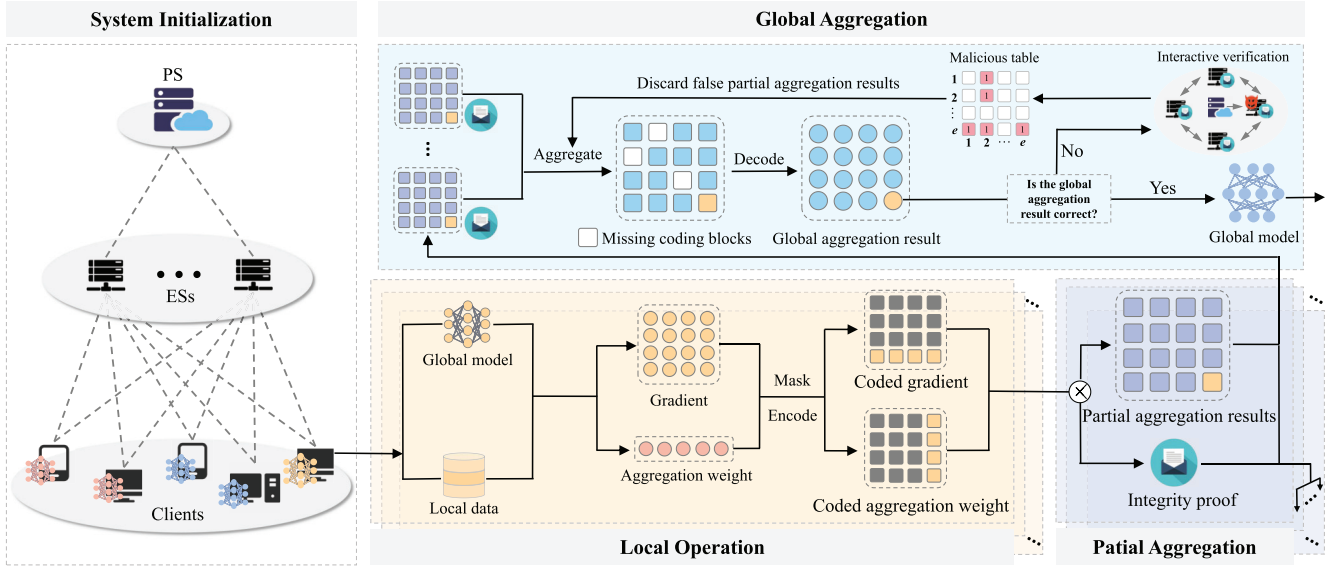


Fig. 3. The workflow of RaSA. In RaSA, three phases are performed after the system initialization.

the PS aggregates the correct partial aggregation results by excluding the malicious partial aggregation results according to S_m^* and decodes to obtain the correct global aggregation result: $\mathbf{W} \times \mathbf{M} = \sum_{i=1}^{k-1} \sum_{j=1}^{k-1} (\tilde{\mathbf{W}} \times \tilde{\mathbf{M}})_{i,j}$.

VI. THEORETICAL ANALYSIS

In this section, we first present the correctness of the proposed one-time masking mechanism, followed by the convergence analysis. We will then revisit our design goals for RaSA and provide a comprehensive analysis.

A. Correctness

Theorem 1 (Correctness) The correctness of RaSA depends on the proposed one-time masking mechanism, which is determined by effectively eliminating aggregated masks and closely related to the linear coding methods.

B. Convergence

Assumption 1: For client $P_i \in [u]$, the loss function $F_i(\theta)$ is Lipschitz smoothness, i.e., for any local model θ and θ' , there exists $F(\theta') - F(\theta) \leq \langle \nabla F(\theta), \theta' - \theta \rangle + \frac{L}{2} \|\theta' - \theta\|_2^2$.

Assumption 2: For client $P_i \in [u]$, the loss function $F_i(\theta)$ is Lipschitz smoothness, i.e., for any local model θ and θ' , there exists $F(\theta') - F(\theta) \geq \langle \nabla F(\theta), \theta' - \theta \rangle + \frac{\mu}{2} \|\theta' - \theta\|_2^2$.

Assumption 3: The variance of the stochastic gradient of client P_i is bounded, and ξ_i^t is the t th round of uniformly random sampling from the local data of client P_i . There are $\mathbb{E} \|\nabla F_i(\theta_i^t, \xi_i^t) - \nabla F_i(\theta_i^t)\| \leq \varrho_i^2$.

Assumption 4: The expected squared norm of stochastic gradients is uniformly bounded, i.e., $\mathbb{E} \|\nabla F_i(\theta_i^t, \xi_i^t)\|^2 \leq B^2$.

Theorem 2: Let F_i^* and F^* be the minimum of F_i and F respectively, and use $Q = F^* - \sum_{i=1}^u w_i F_i^* \geq 0$ to quantify the degree of non-IID data. Let Assumptions 1 to 4 hold and the learning rate $\delta_t = \frac{2}{\varphi t}$. RaSA satisfies

$$\mathbb{E}[F(\bar{\theta})] - F^* \leq \frac{L}{t} \left(\frac{2H}{\varphi^2} + \frac{1}{2} \mathbb{E} \|\theta' - \theta^*\|^2 \right),$$

where $H = \sum_{i=1}^u w_i^2 \varrho_i^2 + 6LQ + 8(E-1)B^2$.

Due to space constraints, the formal proofs of Theorem 1 and Theorem 2 are detailed in our submitted *Supplemental Materials*.

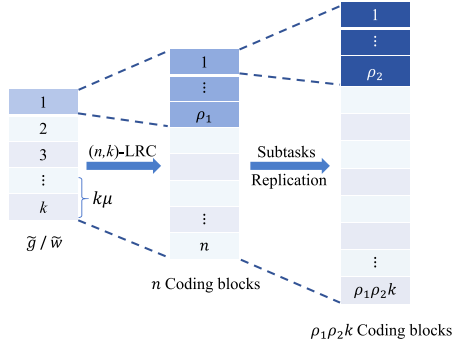
C. Robustness

Theorem 3: The adaptive weights updating algorithm guarantees the goals of accelerating model convergence speed and enhancing the model generalization ability in the non-IID data scenario.

Proof. If the aggregation weight is updated by only considering similar gradients, i.e., ignoring heterogeneous gradients, the model aggregation speed and model generalization ability will be directly affected in the non-IID data scenario. To address this issue, we propose Algorithm 1 by considering the contribution of both similar gradients to global model convergence and heterogeneous gradients to model generalization ability. To make our algorithm more reasonable, we first quantify the non-IID degree of local data from two aspects: gradient direction and the number of data samples, which make a positive contribution to the global model convergence. According to the non-IID degree of local data, the aggregation weight of each client can be adaptively calculated, which helps ensure the model convergence speed and enhances model generalization ability in the non-IID data scenario. Theorem 2 has given the convergence analysis of the proposed protocol. Moreover, the experimental results will further testify to the effectiveness of the proposed Algorithm 1 for non-IID data, as shown in Section VII-B. This concludes the proof.

Theorem 4: The robustness of system heterogeneity is guaranteed by the redundancy rate of (n, k) -code and the repetition degree of the repetition-based secret sharing.

Proof. To address the communication links failure and dropouts, we employ a linear codes followed by a repetition-based secret sharing and then construct an allocation matrix $\mathbf{I}_{(b+1) \times e}$ to distribute secret shares. Specifically, each client

Fig. 4. Process from $\tilde{g}/\tilde{w}$ to $\rho_1 \rho_2 k$ coding blocks.

distributes n shares of the encoded gradient and aggregation weight to e different ESs according to $\mathbf{I}$. Then each ES can obtain $\beta + 1$ shares from each client, and each share of the client will be transmitted to ρ_2 ESs. In this way, we only need L ($L < e$) ESs to assist the global model aggregation in case communication link failures and dropouts happen.

We use a linear (n, k) -code to encode the masked gradients or aggregation weights to realize the one-dimensional coded computing. As long as there are $L = n - k$ non-straggling ESs, the global aggregation result can be decoded, where $n = ek/L$. In essence, each ES should maintain a fraction μ of the masked gradient and aggregation weight, i.e., $k\mu$, where $\mu \in [\frac{1}{e}, 1)$. The lower bound $\frac{1}{e}$ ensures that all shares of the codewords have been submitted to ESs, which is the sufficient condition for decoding. Therefore, the correct decoding depends on the redundancy rate of (n, k) -code $\rho_1 = n/k = e/L$ and the repetition degree $\rho_2 = (n - k)\mu = L\mu$.

Since we integrate the repetition-based secret sharing with the product-coded computing, the redundancy from both (n, k) -code and repetition degree ρ_2 satisfies that $e\mu = \rho_1 \rho_2$. Fig. 4 shows the process from $\tilde{g}/\tilde{w}$ to $\rho_1 \rho_2 k$ coding blocks and the relationship among the above parameters. Due to the iterative decoding characteristic of product-coded computing, less than L non-straggling ESs are sufficient to realize correct decoding. In RaSA, each client encodes the masked local gradient $\tilde{g}_i = [\tilde{g}_{i,1} \ \tilde{g}_{i,2} \ \dots \ \tilde{g}_{i,k}]$ as $C_i = [C_i^{(1)} \ C_i^{(2)} \ \dots \ C_i^{(n)}]$ and encodes the masked weight $\tilde{w}_i = [\tilde{w}_{i,1} \ \tilde{w}_{i,2} \ \dots \ R_i]$ as $S_i = [S_i^{(1)} \ S_i^{(2)} \ \dots \ S_i^{(n)}]^T$. Each ES stores up to $n\mu$ vectors from the $C^{(i)} = \{C^{(j)}\}_{j=1}^n$ and $S^{(i)} = \{S^{(j)}\}_{j=1}^n$, and computes the partial aggregation result $\{S^{(1)}C^{(1)}, S^{(2)}C^{(2)}, \dots, S^{(n\mu)}C^{(n\mu)}\}$. The structure of product-coded matrix multiplication allows the PS to decode the global aggregation result iteratively, which provides more flexibility for the decoding procedure.

When ρ_1 and ρ_2 are fixed, only the partial aggregation results returned by η ($\eta < L$) ESs are required to decode global aggregation results. The parameters η, ρ_1, ρ_2 need to satisfy the following constraint according to the work [53].

$$\binom{e}{\rho_2} - \binom{e-\eta}{\rho_2} \geq \frac{1}{\rho_1} \binom{e}{\rho_2}$$

If the number of dropped ESs is less than $e - \eta$, the PS will receive the matrix:

$$\begin{bmatrix} S^{(1)}C^{(1)} & S^{(2)}C^{(1)} & S^{(3)}C^{(1)} & \dots & \emptyset \\ \emptyset & \emptyset & S^{(3)}C^{(2)} & \dots & S^{(n)}C^{(2)} \\ S^{(1)}C^{(3)} & S^{(2)}C^{(3)} & \emptyset & \dots & S^{(n)}C^{(3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ S^{(1)}C^{(n)} & \emptyset & S^{(3)}C^{(n)} & \dots & \emptyset \end{bmatrix}$$

where $\emptyset$ denotes the missing blocks. Since each row and each column of the aggregation matrix are (n, k) -codes, even if some rows or columns do not reach the decoding recovery threshold, the global aggregation result can be obtained by iterative decoding of the rows and columns. This concludes the proof.

Theorem 5: The robustness against model poisoning attacks from malicious edge servers is guaranteed by the randomness of pseudorandom generators $PRG(\cdot)$, the predictability of pseudorandom functions $PRF(\cdot)$, and the security of SNIP.

We present formal proofs of Theorem 5 and the following Theorem 6, Theorem 7 in the *Supplemental Materials*.

D. Privacy

Theorem 6 (Privacy against colluding edge servers) Any subset of up to $\mathcal{C}$ colluding edge servers learn nothing about the local gradient g_i and aggregation weight w_i of P_i . More precisely, for any $\tau \subset [N]$ with $|\tau| \leq \mathcal{C}$, where $\mathcal{C} = \lfloor \frac{k-1}{\rho_2} \rfloor$, the mutual information must satisfy:

$$I\left(\sum_{e=1}^{\tau} (\tilde{g}_{i,e}, \tilde{w}_{i,e}); (g_i, w_i)_{i \in [u]}\right) = 0.$$

Theorem 7 (Privacy against semi-honest PS) The semi-honest PS can not infer any data privacy of the clients due to the randomness of $PRG(\cdot)$ and the masking number along with the zero-knowledge property of SNIP.

VII. PERFORMANCE EVALUATION

In this section, we evaluate the performance of RaSA under scenarios with non-IID data, heterogeneous systems, and malicious edge servers. We also measure the scalability of RaSA in terms of computational costs and communication overheads.

A. Experimental Setup

Implementation: The experiments are conducted on a system equipped with an 8-core AMD processor, 16GB RAM, and 40GB disk space. We adopt a cloud server configured with an RTX 2080Ti GPU, 62GB RAM, and 100GB disk space for the CNN-based model training of the FL task. RaSA is primarily implemented based on Python via the TensorFlow library. We also invoke a C-based Jerasure library to implement LRC codes and RS codes.

Datasets: We employ five benchmark datasets, FEMNIST, MNIST, Fashion-MNIST, CIFAR-10, and Yahoo! Answers. The FEMNIST dataset has images with 28×28 pixels across 62 distinct classes, including 10-digit classes, 26 lowercase

letter classes, and 26 uppercase letter classes. The FEMNIST training images already exhibit a naturally distributed non-IID pattern across 3,400 clients, and we randomly select 100 clients for our experiments. The MNIST dataset has a 10-class digit image classification dataset and the Fashion-MNIST dataset consists of 10 categories of images of 70,000 fashion products. Both MNIST and Fashion-MNIST consist of 60,000 training images and 10,000 testing images. The CIFAR-10 dataset consists of 60,000 three-channel RGB images, each with a resolution of 32×32 pixels. These 60,000 samples are partitioned into 50,000 training samples and 10,000 test samples. The Yahoo! Answers dataset consists of approximately 1.4 million question-answer pairs in ten categories, of which about 80% are training sets and the rest are test sets. Similar to FLTrust [11], we divide 100 clients into ζ groups if the number of categories in the given dataset is ζ . For instance, we divide 100 clients into 10 groups when using the MNIST dataset. We assign the training images with label $\mathcal{L}$ to group $\mathcal{L}$ with probability $q > 0$, and any other groups get the label- $\mathcal{L}$ training images with probability $\frac{1-q}{\zeta-1}$. Samples are randomly distributed to each client within the same group.

Parameters Settings: We use a parameter q to indicate the non-IID degree of the training data among clients. We note $q=0.1$ as MNIST-0.1, which indicates the local training data are IID among clients. We also set MNIST-0.3 and MNIST-0.6 to simulate non-IID local training data among clients. The same settings apply to Fashion-MNIST, CIFAR-10, and Yahoo! Answers datasets. To simulate a heterogeneous system, we add random delays of $0 \sim 30$ ms to the computations conducted by 100 clients and 10 edge servers. This simulates different levels of straggler effects that could be caused by weak computing powers and intermittent network connections in a real-world FL setup. Furthermore, to simulate unstable network connections, we randomly select some unstable clients, edge servers, or communication links, which would drop out at any time during the training process.

Metrics: We assess classification models' performance by accuracy ($Acc. = \frac{\text{\#correctly classified records}}{\text{\#total records}}$), and measure the accuracy of the classification model by cross-entropy.

Baselines: We compare RaSA with the following schemes from different aspects.

- **FedAvg** [1] is the standard FL aggregation algorithm that does not consider the impact of similar gradients and heterogeneous gradients on model aggregation.
- **FedEdge** [9] prioritizes heterogeneous gradients over similar gradients to exploit the diversity in intermediate models under hierarchical FL architecture.
- **FLTrust** [11] updates the aggregation weights of each client by the server based on the cosine similarity of local gradients and the global gradient.
- **LCM** [36] is a robust SA scheme based on coded computing, which only encodes gradients and sends them to multiple servers to address the unreliable communication links between servers and clients.
- **Schlegel et al.** [50] is based on product-coded computing where multiple clients perform a linear inference $\mathcal{WD}$ on some local data $\mathcal{D}$ given a network-side matrix $\mathcal{W}$. They

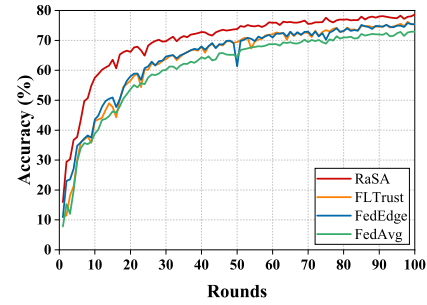


Fig. 5. Comparison of the model performance on the FEMNIST dataset.

adopt Reed-Solomon codes (RS) to encode both $\mathcal{W}$ and $\mathcal{D}$ to achieve iterative decoding of rows and columns.

B. Performance on Non-IID Data

We first evaluate the performance of the proposed adaptive weight updating strategy (Algorithm 1) on the non-IID dataset FEMNIST, and then assess the performance of the algorithm under different setup of non-IID data on MNIST, Fashion-MNIST, CIFAR-10, and Yahoo! Answers.

Effectiveness of Algorithm 1: As shown in Fig. 5, RaSA has advantages over FLTrust [11], FedEdge [9], and FedAvg [1] in both convergence speed and accuracy. For instance, the test accuracy is 69.7%, 63.7%, 64.6%, and 60.1% for RaSA, FLTrust, FedEdge, and FedAvg at the 30th training round. When the model converges, the accuracy is 78.6%, 75.3%, 75.4%, 72.9% for RaSA, FLTrust, FedEdge, and FedAvg respectively.

Fig. 6 shows that the convergence speed of the global model slows down and the model accuracy decreases when the non-IID degree of datasets increases. The model accuracy of RaSA is 88.5% and 81.2% for MNIST-0.3 and MNIST-0.6 at the 30th training round. If the model accuracy is required to achieve 90% on MNIST-0.3 and MNIST-0.6, it needs around 34 and 53 training rounds respectively. The empirical results show that RaSA can maintain a high accuracy and fast convergence speed under different non-IID data scenarios. As shown in Fig. 6(a)(d)(g)(j), all schemes exhibit similar convergence performance across the four datasets when the clients' data is IID. The performance of RaSA is slightly better than FLTrust and FedEdge, and significantly better than FedAvg. For the non-IID data settings, such as MNIST-0.6 (shown in Fig. 6(c)), when the model accuracy reaches 80%, it takes 29, 31, and 33 training rounds for RaSA, FedEdge and FLTrust respectively. When using the Fashion-MNIST-0.3 dataset, it takes 45, 49, 54, and 78 rounds for RaSA, FedEdge, FLTrust, and FedAvg to achieve the model accuracy of 70%. When using CIFAR-10 or Yahoo! Answers, RaSA shows better performance than the baselines. As shown in Fig. 6(h) (CIFAR-10-0.3), it takes 24, 51, and 81 rounds respectively for RaSA, FedEdge, and FLTrust to achieve a model accuracy of 60%. In contrast, even after 100 rounds of training, the model accuracy of FedAvg fails to reach 60%.

Moreover, we compare RaSA with baselines to evaluate the generalization ability of the trained models on different non-IID settings. The non-IID degree q affects the trained

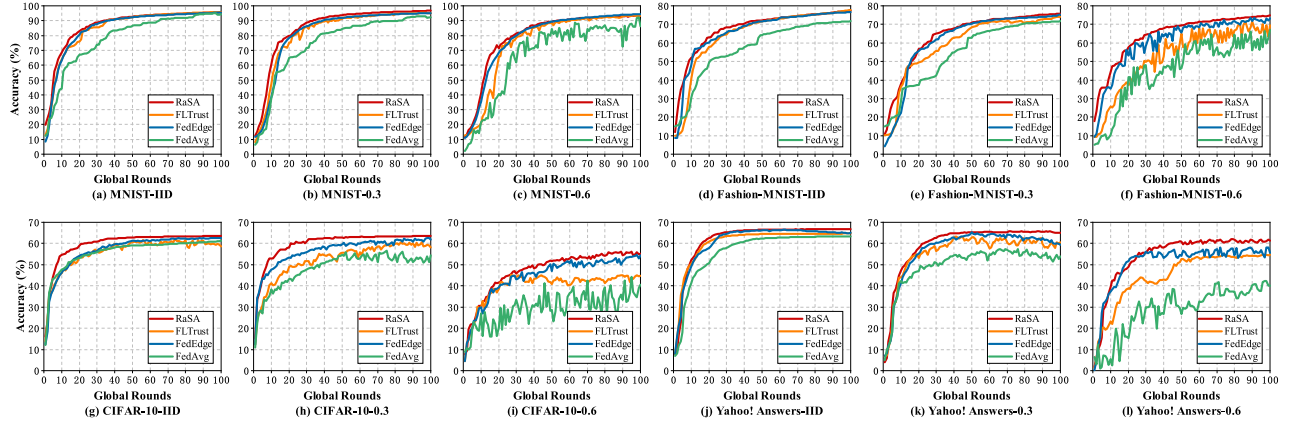


Fig. 6. Comparison of the test accuracy and convergence speed under different non-IID settings.

TABLE III
PERFORMANCE OF GLOBAL MODEL GENERALIZATION ON
FOUR DATASETS WITH NON-IID DEGREE q

Schemes	MNIST		Fashion-MNIST		CIFAR-10		Yahoo! Answers	
	$q=0.3$	$q=0.6$	$q=0.3$	$q=0.6$	$q=0.3$	$q=0.6$	$q=0.3$	$q=0.6$
FedAvg [1]	0.922	0.862	0.719	0.669	0.545	0.405	0.539	0.401
FedEdge [9]	0.952	0.943	0.767	0.731	0.618	0.545	0.592	0.557
FLTrust [11]	0.951	0.934	0.776	0.694	0.582	0.448	0.601	0.547
RaSA (Ours)	0.968	0.936	0.777	0.749	0.635	0.551	0.651	0.614

model accuracy of all schemes. Table III shows the test accuracy at round 100 for different schemes with different q . A higher q leads to a lower accuracy of the model. The accuracy varies across MNIST, Fashion-MNIST, CIFAR-10, and Yahoo! Answers datasets. We find that RaSA obtains a higher accuracy on different settings compared to FLTrust, FedEdge, and FedAvg. That is due to the adaptive weight updating strategy which considers the contribution of both similar gradients and heterogeneous gradients to the convergence speed and the generalization ability of the global model. While FLTrust neglects the impact of heterogeneous gradients on the global model, and FedEdge prioritizes the contribution of heterogeneous gradients to the global model.

In particular, we measure the computational cost of the weight updating algorithm by comparing RaSA to FLTrust and FedEdge. As depicted in Table IV, when the number of clients is set to be 50, 80, and 100, the time cost for weight updating of RaSA is 0.0277s, 0.0465s, and 0.0533s, respectively. The time cost increases with the number of clients. RaSA is approximately four orders of magnitude faster than FedEdge, and this gap widens with the increasing number of clients. Because FedEdge is required to compute the distances between it and all other gradients in order to calculate the aggregation weight of one gradient. This process takes most of the computational cost.

C. Performance Under Heterogeneous System

To evaluate the performance of RaSA against the heterogeneous system, we categorize the causes of the system heterogeneity into three types: 1) Dropped clients: Clients may drop out during the local operation phase or after sending

TABLE IV
COMPUTATIONAL COST OF WEIGHT UPDATING

Number of Clients	FedEdge	FLTrust	RaSA
50	337.2609 s	0.0276 s	0.0277 s
80	910.4269 s	0.0490 s	0.0465 s
100	1932.2847 s	0.0563 s	0.0533 s

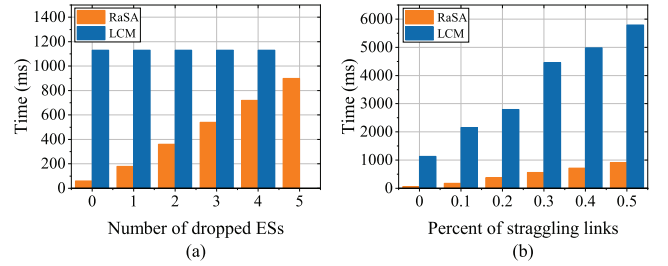


Fig. 7. Comparison of the computational cost of decoding using different coding methods.

messages. 2) Dropped ESs: The ESs may drop out during the partial aggregation phase, failing to send the partial aggregation results to the PS. 3) Straggling links: The communication links from the clients to the ESs and from the ESs to the PS may be lost. Since RaSA need not select participating clients in advance, clients can drop out at any time, which will not affect the model aggregation. Hence, we only evaluate the performance of RaSA against the dropped ESs and Straggling links.

Effectiveness of Product-Coded Computing: RaSA takes advantage of the product-coded computing based on LRC codes to address the system heterogeneity. We evaluate the performance of RaSA in product-coded computation. The time cost of the gradient encoding is only related to the gradient size. When the gradient size is 10000, 30000, 50000, 70000, and 90000, it takes 101.3ms, 247.8ms, 385.7ms, 535.1ms, and 662.9ms to encode the gradients in RaSA. The encoding time for the aggregation weights is trivial. As for the decoding performance, we compare RaSA to LCM [36] which is based on Lagrange coded computing. As shown in Fig. 7(a), the

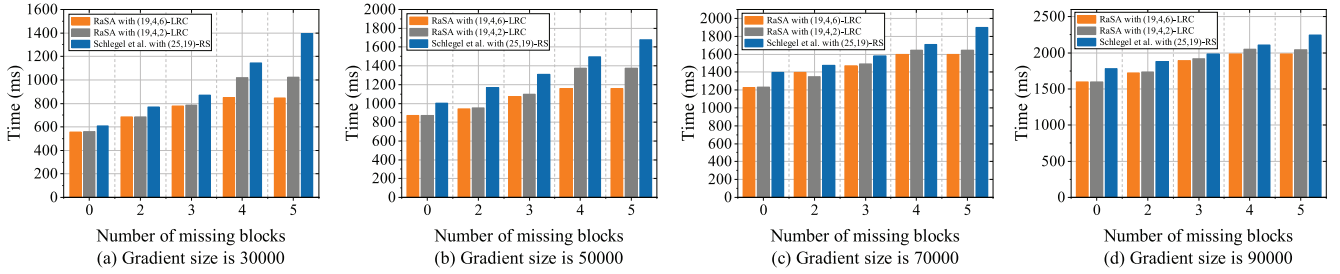


Fig. 8. The impact of straggling links on the computational cost.

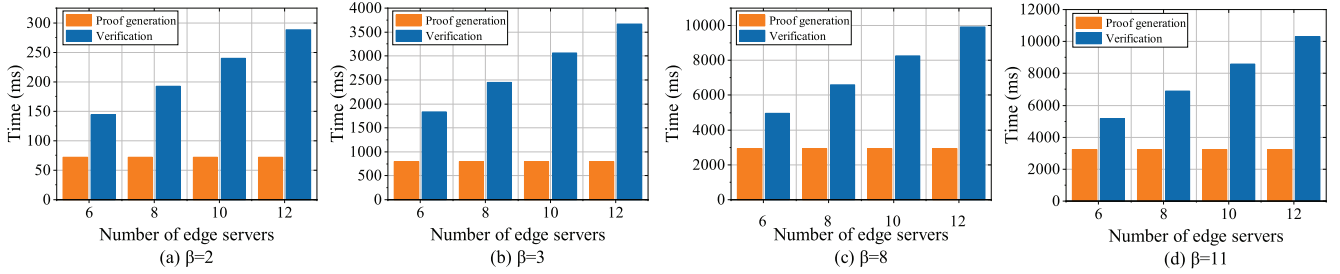


Fig. 9. Computational cost on the proof generation and verification.

number of dropped ESs has a greater impact on LCM than RaSA. Although the time cost of RaSA increases as the number of dropped ESs increases, it is still much less than that of LCM. In particular, LCM can not handle the decoding task when the number of the dropped ESs is greater than five.

We also evaluate the performance of RaSA against the straggling links. As shown in Fig. 7(b), the decoding time of RaSA and LCM increases with the percentage of straggling links, but the computational efficiency of RaSA is significantly better than that of LCM. When the percentage of straggling links is 0%, 10%, 20%, 30%, 40%, and 50%, the decoding time of RaSA is 62ms, 175ms, 374ms, 559ms, 720ms, and 913ms respectively. The empirical results show that the decoding performance of RaSA is $6.35\times$ faster than that of LCM. Because the matrix of the aggregation results $S \times C \in GF(q)^{n \times n}$ of RaSA is a special matrix where each row and column is encoded by LRC codes. If there is only one dropped ES, only 3 coding blocks ($\beta + 1$) are missing. Therefore, only intra-group decoding is enough, which costs around 60ms. In case the number of dropped ESs increases and more blocks will be lost, the iterations decoding plays an important role in decreasing the computational cost of the decoding.

Comparison With RS-Based Scheme: We compare RaSA to the RS-based scheme [50] under the straggling links scenarios as shown in Fig. 8. The straggling links directly lead to missing blocks in the matrix $S \times C$. Since the total time of encoding and decoding in RaSA is related to the number of missing blocks and the gradient size, we conduct extensive experiments under different settings. Specifically, when the gradient size is 30,000 and the number of missing blocks ranges from 0 to 5, it takes 557ms, 684ms, 779ms, 849ms, and 995ms respectively for RaSA based on (19,4,6)-LRC code. By contrast, the time for RS-based scheme [50] increases by 51ms, 85ms, 93ms, 297ms, and 399ms respectively. Because the adopted LRC code allows for intra-group decoding when there are only a few missing blocks, while global decoding is employed when a larger number of blocks are lost.

TABLE V
THE IMPACT OF GRADIENT SIZE ON COMPUTATIONAL COSTS

Gradient Size	Local Operation	Partial Aggregation	Global Aggregation
10000	135.3 ms	103.6 ms	317.8 ms
30000	283.7 ms	149.9 ms	477.2 ms
50000	421.1 ms	182.1 ms	618.8 ms
70000	565.4 ms	224.0 ms	734.7 ms
90000	698.3 ms	267.5 ms	917.9 ms

D. Performance Under Malicious Edge Servers

To resist malicious edge servers, it is imperative to realize the verification for the local aggregation process and the global model in RaSA, which will bring in extra computational costs and communication overheads.

1) Cost With No Malicious ES: In RaSA, every ES needs to generate integrity proofs of the partial aggregation process and secretly share them with others. If there is no malicious edge server, the global aggregation result will pass the verification conducted by the PS, and there is no need to verify the integrity proofs. Hence, we first evaluate the computational cost of global aggregation verification, which is related to the number of clients. When the number of clients is 100, 500, 1000, and 1500, it takes 1.017ms, 3.895ms, 9.713ms, and 11.472ms respectively to verify the correctness of the global aggregation result in RaSA. This is primarily attributed to generating pseudorandom numbers with PRF and PRG.

As shown in Fig. 9, we also evaluate the computational cost of the integrity proof generation for ESs. The time cost of the proof generation is related to the dimension of the allocation matrix $I_{(\beta+1) \times e}$. The parameter $\beta = \lceil \frac{\rho_1 \rho_2 k}{e} - 1 \rceil$, where $\rho_1 = \frac{n}{k}$. Thus, when the n and e are fixed, β only depends on ρ_2 . The time cost of proof generation increases linearly with the increase of β but remains constant with the increase of the number of edge servers.

2) Cost With Malicious ESs: If an ES is maliciously manipulated, RaSA needs to perform additional verification of the partial aggregation process. We also evaluate the computa-

TABLE VI
COMMUNICATION OVERHEADS ON DIFFERENT GRADIENT SIZE AND REPLICATION RATE

Gradient size	10000	20000	30000	40000	50000	60000	70000	80000	90000
Coded local gradients /(KB)	118.52	246.05	356.34	492.11	615.24	737.89	851.45	984.21	1071.79
Partial aggregation results /(KB)	125.31	250.43	371.89	501.31	631.47	742.93	873.29	978.63	1167.82
Integrity proofs /(KB)	245.74	496.48	726.73	993.64	1249.31	1482.75	1733.74	1966.42	2243.84
* ESs =10, and the coding method is (29, 19)-LRC.									
Repetition degree ρ_2	1	2	3	4	5	6	7	8	9
Coded local gradients /(MB)	1.07	2.65	3.29	4.29	5.33	6.89	7.36	8.51	9.49
Partial aggregation results /(MB)	1.17	2.71	3.31	4.31	5.47	6.93	7.49	8.33	9.82
Integrity proofs /(MB)	2.31	5.42	6.71	8.74	11.49	14.32	15.33	16.46	19.49

*The gradient size is 93088 for CNN and the coding method is (120, 80)-LRC.

tional cost of the integrity proof verification for ESs. As shown in Fig. 9, the verification time increases not only linearly with β but also with the number of edge servers. Moreover, the verification time is significantly higher than the proof generation time. For instance, when $\beta = 2$ and the number of edge servers is 6, 8, 10, and 12, the proof generation time is approximately 72ms, while the verification time is 144ms, 192ms, 240ms, and 288ms, respectively. Because proofs are generated only once, however, the number of interactive verifications increases as the number of edge servers increases. In addition, since the size of the matrix received by the edge server increases with β , it leads to an increased number of multiplication gates in the *Valid*(·) circuit, which incurs higher overhead for proof verification.

E. Computational Cost

We evaluate the computational cost of different phases of RaSA, including system initialization, local operation, partial aggregation, and global aggregation. Table V shows the computational cost of RaSA based on (19, 4, 6)-LRC with 100 clients, and 10 ESs in case of no stragglers. In the initialization phase, the PS only needs to select some parameters to publish, and the computational cost is negligible.

Cost at Local Operation: The local operation mainly involves weight updating, masking, and encoding. The computational cost of weight updating and masking is independent of the gradient size, approximately 30ms and 5ms respectively. However, as the number of gradients increases, the dimension of the matrix to be encoded also increases, leading to a higher computational cost of coding.

Cost at Partial Aggregation: The partial aggregation is executed by edge servers, which involves the matrix multiplication of the coded gradients and the coded aggregation weights and the integrity proof generation. Since the dimension of coded gradients is influenced by the gradient size. Therefore, the computational cost of matrix multiplication increases with the gradient size. For the integrity proof generation, it takes about 75ms for each ES to generate a proof, and the computational cost of which is irrelevant to the gradient size.

Cost at Global Aggregation: The global aggregation mainly includes decoding, global aggregation result verification, partial aggregation process verification, and weight updating parameters aggregation. Since the partial aggregation process verification is only needed when there exist malicious edge servers. For clarity, we assume there is one malicious edge server. The computational cost of decoding operations is directly proportional to the gradient size. When the gradient

size is set to be 10000, 30000, 50000, 70000, and 90000, it costs 57.8ms, 217.2ms, 358.8ms, 474.7ms, and 657.9ms respectively to decode the masked global aggregation result in RaSA. While the cost of the other three operations remains unaffected by the number of gradients, i.e., the cost of the global aggregation verification, partial aggregation process verification, and weight updating parameters aggregation is around 1ms, 240ms, and 5ms respectively.

F. Communication Overhead

The communication overheads of RaSA mainly include the transmission of coded local gradients and coded aggregation weights from clients to ESs, the exchange of integrity proofs among multiple ESs, and the submission of partial aggregation results from ESs to the PS. We evaluate the communication overheads of RaSA through the increasing gradient size and the varying repetition degree ρ_2 . We use (29, 19)-LRC codes and the MNIST datasets, and the number of the edge servers is set to 10. Table VI shows the experimental results.

The size of coded local gradients depends on the gradient size, and the partial aggregation result is the matrix multiplication of coded gradients and the coded weights, therefore the size of the partial aggregation result is also related to the gradient size. Since the integrity proof of RaSA consists of coded local gradients, coded aggregation weights, and the partial aggregation result, the size of the integrity proof also depends on the gradient size. As shown in Table VI, when repetition degree ρ_2 grows, the communication overheads of RaSA scale with ρ_2 linearly under the fixed gradient size.

VIII. CONCLUSION

In this paper, we proposed RaSA, a robust and adaptive secure aggregation protocol that enhances the privacy protection and robustness of the secure aggregation in an edge-assisted hierarchical FL system. Compared to prior works, RaSA provides a brand-new solution to address data and system heterogeneity with the purpose of accelerating the model convergence rate and enhancing the model generalization ability, and RaSA simultaneously defends against active attacks from the malicious edge servers without impacting the model accuracy. The security analysis proved the security of RaSA, and extensive experiments demonstrated that RaSA achieves strong privacy protection and robustness of model aggregation at a reasonable computational cost. Particularly, RaSA improves the dropout tolerance and reduces the decoding delay compared to the existing coding-based SA protocol. RaSA can also be combined with our prior work [40] to address the malicious client issue. We leave as future work

of how to choose a more appropriate boundary angle γ and optimize the calculation of weight updating factors. Moreover, the coding block allocation has a repetition degree ρ_2 in RaSA, which results in extra communication overheads. Hence, we also leave as a promising problem of designing a better allocation matrix $\mathbf{I}$ to achieve the maximum trade-off between communication overheads and decoding delay.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, A. Singh and J. Zhu, Eds., Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [2] A. Rodio, F. Faticanti, O. Marfoq, G. Neglia, and E. Leonardi, "Federated learning under heterogeneous and correlated client availability," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, May 2023, pp. 1–10.
- [3] H. Wu and P. Wang, "Fast-convergent federated learning with adaptive weighting," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 4, pp. 1078–1088, Dec. 2021.
- [4] Y. Liao, Y. Xu, H. Xu, L. Wang, and C. Qian, "Adaptive configuration for heterogeneous participants in decentralized federated learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, New York, NY, USA, May 2023, pp. 1–10.
- [5] R. Tandon, Q. Lei, A. Dimakis, and N. Karampatziakis, "Gradient coding: Avoiding stragglers in distributed learning," in *Proc. 34th Int. Conf. Mach. Learn.*, Sydney, NSW, Australia, Jul. 2017, pp. 3368–3376.
- [6] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2017, pp. 1175–1191.
- [7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 14774–14784.
- [8] J. Hu et al., "Shield against gradient leakage attacks: Adaptive privacy-preserving federated learning," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1407–1422, Apr. 2024.
- [9] K. Wang, Q. He, F. Chen, H. Jin, and Y. Yang, "FedEdge: Accelerating edge-assisted federated learning," in *Proc. ACM Web Conf.*, Singapore, Apr. 2023, pp. 2895–2904.
- [10] Z. Chai, Y. Chen, A. Anwar, L. Zhao, Y. Cheng, and H. Rangwala, "FedAT: A high-performance and communication-efficient federated learning system with asynchronous tiers," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, New York, NY, USA, Nov. 2021, pp. 1–16.
- [11] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2021, pp. 1–18.
- [12] C. P. Wan and Q. F. Chen, "Robust federated learning with attack-adaptive aggregation," in *Proc. FTL-IJCAI'21*, Montreal, QC, Canada, 2021, pp. 1–22.
- [13] S. Hou, S. Li, T. Jahani-Nezhad, and G. Caire, "PriRoAgg: Achieving robust model aggregation with minimum privacy leakage for federated learning," 2024, *arXiv:2407.08954*.
- [14] J. Feng, L. Liu, Q. Pei, and K. Li, "Min-max cost optimization for efficient hierarchical federated learning in wireless edge networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2687–2700, Nov. 2022.
- [15] Q. He et al., "A game-theoretical approach for mitigating edge DDos attack," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2333–2348, Jul. 2022.
- [16] B. Li et al., "Cooperative assurance of cache data integrity for mobile edge computing," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4648–4662, 2021.
- [17] W. Kim, S. Kruglik, and H. Mao Kiah, "Verifiable coded computation of multiple functions," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8009–8022, 2024.
- [18] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *Proc. 14th USENIX Symp. Networked Syst. Design Implement.*, Boston, MA, USA, 2017, pp. 259–282.
- [19] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learn. Represent.*, Addis Ababa, Ethiopia, 2020, pp. 1–26.
- [20] S. Chen, C. Shen, L. Zhang, and Y. Tang, "Dynamic aggregation for heterogeneous quantization in federated learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6804–6819, Oct. 2021.
- [21] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "A novel framework for the analysis and design of heterogeneous federated learning," *IEEE Trans. Signal Process.*, vol. 69, pp. 5234–5249, 2021.
- [22] Z. Zhao and G. Joshi, "A dynamic reweighting strategy for fair federated learning," in *Proc. ICASSP - IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Singapore, May 2022, pp. 8772–8776.
- [23] Q. Ma, Y. Xu, H. Xu, Z. Jiang, L. Huang, and H. Huang, "FedSA: A semi-asynchronous federated learning mechanism in heterogeneous edge computing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3654–3672, Dec. 2021.
- [24] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. 15th USENIX Symp. Operat. Syst. Design Implement. (OSDI)*, Jul. 2021, pp. 19–35.
- [25] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.
- [26] Y. Ma, J. Woods, S. Angel, A. Polychroniadou, and T. Rabin, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proc. IEEE Symp. Security Privacy (SP)*, San Francisco, CA, USA, Jun. 2023, pp. 477–496.
- [27] J. So, B. Güler, and A. S. Avestimehr, "Turbo-aggregate: Breaking the quadratic aggregation barrier in secure federated learning," *IEEE J. Sel. Areas Inf. Theory*, vol. 2, no. 1, pp. 479–489, Mar. 2021.
- [28] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2020.
- [29] B. Sasidharan and A. Thomas, "Coded gradient aggregation: A tradeoff between communication costs at edge nodes and at helper nodes," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 3, pp. 761–772, Mar. 2022.
- [30] R. Schlegel, S. Kumar, E. Rosnes, and A. G. I. Amat, "CodedPaddedFL and CodedSecAgg: Straggler mitigation and secure aggregation in federated learning," *IEEE Trans. Commun.*, vol. 71, no. 4, pp. 2013–2027, Apr. 2023.
- [31] H. Hu, Y. Wu, Y. Shi, S. Li, C. Jiang, and W. Zhang, "Communication-efficient coded computing for distributed multi-task learning," *IEEE Trans. Commun.*, vol. 71, no. 7, pp. 3861–3875, Jul. 2023.
- [32] J. So et al., "Lightsecagg: A lightweight and versatile design for secure aggregation in federated learning," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 4, 2022, pp. 694–720.
- [33] T. Jahani-Nezhad, M. A. Maddah-Ali, S. Li, and G. Caire, "SwiftAgg+: Achieving asymptotically optimal communication loads in secure aggregation for federated learning," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 977–989, Apr. 2023.
- [34] Y. Zhao and H. Sun, "Information theoretic secure aggregation with user dropouts," *IEEE Trans. Inf. Theory*, vol. 68, no. 11, pp. 7471–7484, Nov. 2022.
- [35] J. Zhu and S. Li, "Generalized Lagrange coded computing: A flexible computation-communication tradeoff," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Espoo, Finland, Nov. 2022, pp. 832–837.
- [36] K. Liang, S. Li, M. Ding, and Y. Wu, "Multi-server secure aggregation with unreliable communication links," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2023, pp. 2560–2565.
- [37] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, and A. Hithnawi, "RoFL: Robustness of secure federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 453–476.
- [38] A. R. Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "EIFFeL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Los Angeles, CA, USA, Nov. 2022, pp. 2535–2549.
- [39] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
- [40] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, "TMT-FL: Enabling trustworthy model training of federated learning with malicious participants," *IEEE Trans. Depend. Sec. Comput.*, early access, Dec. 23, 2024, doi: [10.1109/TDSC.2024.3521377](https://doi.org/10.1109/TDSC.2024.3521377).

- [41] S. Prakash, A. Reiszadeh, R. Pedarsani, and A. S. Avestimehr, "Hierarchical coded gradient aggregation for learning at the edge," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Los Angeles, CA, USA, Jun. 2020, pp. 2616–2621.
- [42] B. Sasidharan and A. Thomas, "Coded gradient aggregation: A tradeoff between communication costs at edge nodes and at helper nodes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Melbourne, VIC, Australia, Jul. 2021, pp. 2286–2291.
- [43] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1514–1529, Mar. 2018.
- [44] K. Lee, C. Suh, and K. Ramchandran, "High-dimensional coded matrix multiplication," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Aachen, Germany, Jun. 2017, pp. 2418–2422.
- [45] C. Huang et al., "Erasure coding in windows Azure storage," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2012, pp. 15–26.
- [46] J. Håstad, R. Impagliazzo, L. A. Levin, and M. Luby, "A pseudorandom generator from any one-way function," *SIAM J. Comput.*, vol. 28, no. 4, pp. 1364–1396, Jan. 1999.
- [47] O. Goldreich, S. Goldwasser, and S. Micali, "How to construct random functions," *J. ACM*, vol. 33, no. 4, pp. 792–807, Aug. 1986.
- [48] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.
- [49] S. El Rouayheb and K. Ramchandran, "Fractional repetition codes for repair in distributed storage systems," in *Proc. 48th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2010, pp. 1510–1517.
- [50] R. Schlegel, S. Kumar, E. Rosnes, I. Amat, and A. Graell, "Privacy-preserving coded mobile edge computing for low-latency distributed inference," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 3, pp. 788–799, Mar. 2022.
- [51] H. Yan et al., "RECESS vaccine for federated learning: Proactive defense against model poisoning attacks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1–11.
- [52] S. Li, D. Yao, and J. Liu, "FedVS: Straggler-resilient and privacy-preserving vertical federated learning for split models," in *Proc. Int. Conf. Mach. Learning.*, Jan. 2023, pp. 20296–20311.
- [53] J. Zhang and O. Simeone, "Improved latency-communication trade-off for map-shuffle-reduce systems with stragglers," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Brighton, U.K., May 2019, pp. 8172–8176.



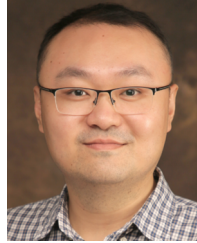
Lingling Wang (Member, IEEE) received the Ph.D. degree from Harbin Engineering University, Harbin, China, in 2008. She was sponsored by China Scholarship Council (CSC) as a Visiting Scholar with Wilfrid Laurier University, Waterloo, ON, Canada, in 2019. She is currently an Associate Professor with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. Her research interests include data security, AI security, edge computing, and applied cryptography.



Mei Huang received the B.E. degree from Qingdao University of Science and Technology, Qingdao, China, in 2022, where she is currently pursuing the master's degree with the School of Information Science and Technology. Her research interests include federated learning and privacy protection.



Zhengyin Zhang received the B.E. degree from Shandong Xiehe University, Jinan, China, in 2022. He is currently pursuing the master's degree with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. His research interests include federated learning and privacy protection.



Meng Li (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from Beijing Institute of Technology, China, in 2019. He was sponsored by the ERCIM's Alain Bensoussan Fellowship Program to conduct post-doctoral research. He was sponsored by CSC for joint Ph.D. studies with the University of Waterloo and Wilfrid Laurier University. He is currently an Associate Professor with the School of Computer Science and Information Engineering, Hefei University of Technology, China. His research interests include data security, privacy preservation, applied cryptography, and the Internet of Vehicles. He is a Senior Member of CIE, CIC, and CCF. He is an Associate Editor of IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, and IEEE INTERNET OF THINGS JOURNAL.



Jingjing Wang (Member, IEEE) received the Ph.D. degree in computer application technology from the Ocean University of China, Qingdao, China, in 2012. She was a Visiting Professor with The University of British Columbia from 2014 to 2015. She is currently a Professor with the School of Information Science and Technology, Qingdao University of Science and Technology. Her research interests include wireless communications, bionic communications, and secure data transmission.



Keke Gai (Senior Member, IEEE) received the Ph.D. degree in computer science from Pace University, New York, NY, USA. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. He has published five books and more than 180 peer-reviewed journal/conference papers, including ten best paper awards. His research interests include cyber security, blockchain, privacy-preserving computation, decentralized identity, and artificial intelligence security. He serves as the Editor-in-Chief for journal *Blockchains* and an Associate Editor for a number of decent journals, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING and *Journal of Parallel and Distributed Computing*.